

THE GRUVS

Product, Architecture, Security and Roadmap · Version 2.5

Restructured from version 2.4 and checked line by line against the code on 5 October 2026. Every feature now shows what is really built, what is partly there, and what only looks finished. Proposals that would be unsafe as written have been redesigned.

What changed from version 2.4

1. **Every feature has a real status**, taken from the code rather than the plan (section 5).
2. **The architecture matches what's deployed**: one Expo app for web and Android, with the website served from a DigitalOcean server. It isn't Next.js on Vercel.
3. **Six new parts**: database and operations, money and payments, discovery and SEO, safety, a decisions page, and a work log.
4. **Four proposals rewritten so they're safe**: Touch Down checks, offline passes, flyer collection (no Instagram scraping), and cross-app logins.
5. **The roadmap is re-ordered**: unblock, then trust, then the live night, then money, then native apps and the ecosystem.

How to read the status labels

Works	Built, and backed by the server.
Partial	Exists, but part of it is missing.
Demo only	Looks finished, but the data is simulated or stays on one phone. Other users never see it.
Not started	No code yet.
Blocked	Needs a decision or an outside account first.
Not reviewed	Exists; not checked in this pass.

Source: TEARN1/The_Gruvs · branch `claude/future-failure-analysis-df20ej` (merged with `main` at `ff486c1`) · Supersedes v2.4

Contents

- 1. Where things stand
- 2. The ecosystem: Live · Play · Work
- 3. Architecture, as built
- 4. Product structure
- 5. Feature register: every feature and its real status
- 6. Widgets and glanceable surfaces
- 7. Security, trust and privacy
- 8. Database and operations
- 9. Money and payments
- 10. Discovery and SEO
- 11. Growth and revenue
- 12. Roadmap
- 13. Decisions needed from you
- 14. Appendix: work completed on this branch

1. Where things stand

The Gruvs is a real-time nightlife and social coordination app for South African cities. Its promise is **ground truth**: who is really at a venue, what's playing, how long the queue is, and how everyone gets home. It's the **Play** part of a Live · Play · Work ecosystem with The Resident (housing) and Excellency Academy (skills).

Area	State on 5 October 2026
Website / PWA	Live at thegruvs.com. It's an Expo (React Native for Web) build served by nginx on a DigitalOcean server.
Android	Built with EAS. Not yet published on Google Play.
iOS	Not started. Live Activities and Dynamic Island depend on it.
Database	Supabase (Postgres with PostGIS). It was paused when last checked (the free tier pauses after about 7 days without activity). Move to the paid tier at launch.
Code	The work branch is merged with <code>main</code> and is 21 commits ahead (including this document), waiting for a pull request.
Quality gates	1,031 automated tests and a clean lint. Database CI builds the schema, applies every migration twice and runs 5 security tests. Secret scanning blocks leaked keys.

The features, by status (38 tracked)



The biggest finding in this review

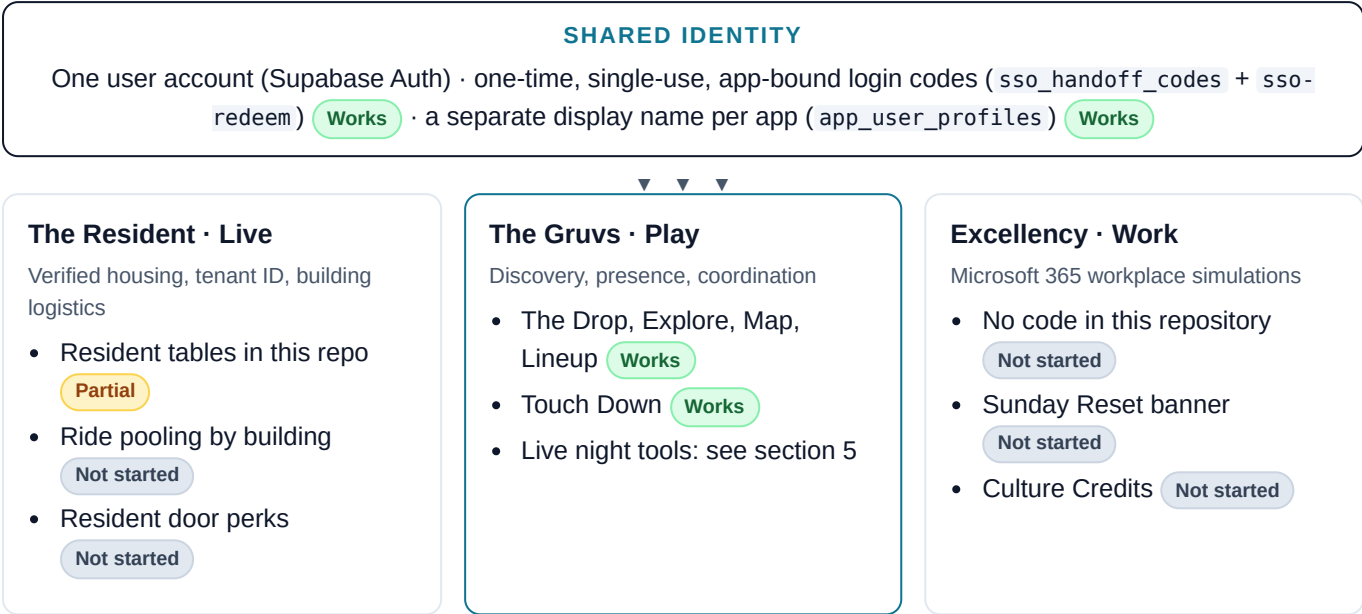
The 9 commits on `main` from 2 to 4 October added about 12,600 lines and a lot of great-looking screens. **10 of those features are demos:** they save to the phone only, use built-in numbers, or try to save to tables that don't exist. Some also tell users things that aren't true: "cryptographically vaulted", "Squad notified", "VERIFIED". The safety panel has been corrected. The rest should be hidden or labelled as previews until they're real. Every push to `main` deploys the website, so **these demos are live on thegruvs.com now**. The app already has the switch to hide them (`feature()` in `launchConfig.js`).

The five things that matter most right now

1. **Restore the database and merge the branch.** Nothing below reaches users until both are done.
2. **Hide or label the demo features**, starting with tickets, money and safety.
3. **Choose a payment provider.** Every money feature waits on it.
4. **Make Touch Down trustworthy** with a rotating door code (section 7.2). It's the heart of "ground truth".
5. **Give Google real pages** for events and cities (section 10).

2. The ecosystem: Live · Play · Work

All three apps share **one Supabase project and one set of user accounts**. A person signs in once and can hop between apps with a one-time code.



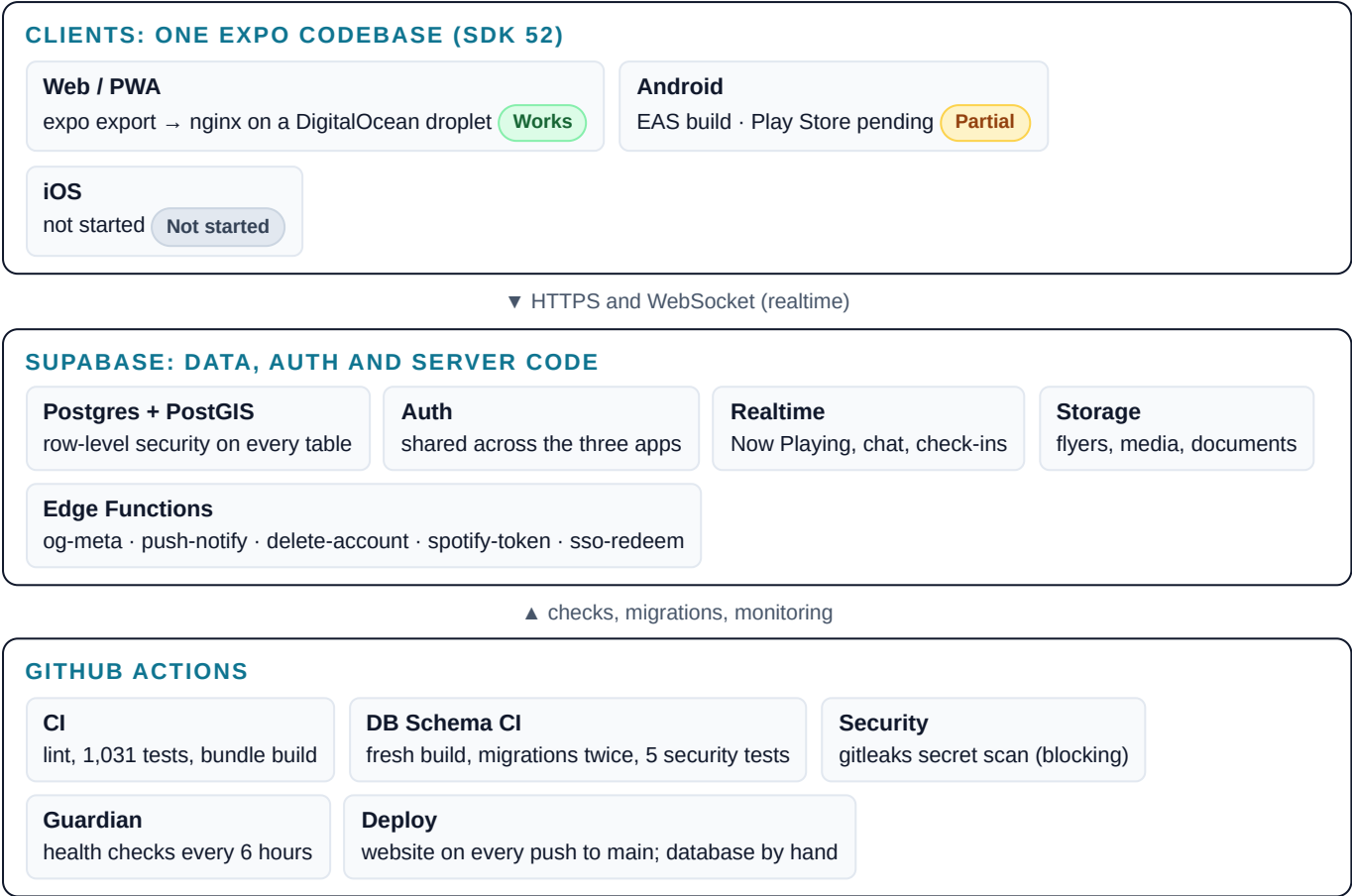
Rules that keep the ecosystem safe

- Separation comes from database rules, not from headers.** The app sends an `x-app-id` header, but the app sets it itself, so anyone can change it. It's fine as a label. It must never decide who can see what. Each app's tables need their own row-level rules.
- Sharing data between apps needs consent.** Under POPIA, using someone's Resident data in The Gruvs (or passing Gruvs users to Excellency as leads) needs a clear opt-in for that purpose.
- Cross-app rewards are money-like.** Culture Credits earned on Excellency and spent on The Gruvs need the same server-only ledger rules as the wallet (section 9). Never let a phone write a balance.

Integration blueprints from v2.4, with status

Blueprint	Status	How to build it safely
Campus-to-Club Ride Pooler	Partial	Build on the existing Carpool Board. Match by "same verified building" without ever showing where anyone lives. Opt-in only, shown only to verified residents of that building.
Verified housing on the map	Not started	A map layer from The Resident's accredited listings. Show the building, never the tenants.
Resident VIP door perks	Not started	Needs venue agreements. The door scanner checks the Resident pass on the server.
Sunday Reset banner	Not started	Needs Excellency to exist and expose its challenges. Time-based placement is fine for this one.
Skill-to-Chill credits	Not started	A server ledger shared across apps. Excellency's server grants the credits; The Gruvs' server spends them.
"Verified Pro" Vibe Card badge	Not started	Set only by Excellency's server after a certification, never from the phone.

3. Architecture, as built



4. Product structure

Navigation: today and proposed

Today: 8 tabs (Reels is hidden in launch mode)

The Drop

Reels

Explore

Map

Lineup

Linked Up

Pings

Vibe Card

▼ proposed **Not started**

Radar
The Drop · Explore · Map · Lineup, as filters of one discovery surface

Live
Shows when you're Locked In tonight or near the venue: Touch Down, On Decks, crowd, queue, crew, ride, safety

Passport
Vibe Card, tickets, wallet, history

Linked Up (chats) and Pings become icons in the header: messaging is how crews coordinate, so it can't be buried.

Planning mode and Live mode

v2.4 switched modes by the clock (weekdays plan, weekends live). Events also happen on weekdays, so **switch on context instead**: Live mode when an event you're Locked In to starts within 3 hours, or when you're inside a venue's radius. Planning mode otherwise. The Sunday Reset slot can stay time-based.

From v2.4	Status	Note
Locked In → Touch Down pipeline	Works	RSVP, then verified presence. Already the core loop.
One primary action per event card	Not started	Cards still carry many actions. Keep [Lock In] or [Touch Down]; move the rest into the event page.
Remove open social feeds	Partial	Reels is hidden in launch mode. The Drop stays as a curated feed.
No background GPS	Works	No background location tracking found. Keep it foreground-only: Google Play reviews "all the time" location strictly.
Offline-first	Partial	Check-ins are queued and replayed; tickets are cached for display. Signed passes (7.3) finish it.
AVIF/WebP flyers	Partial	WebP is used in places. Convert on upload with one pipeline.

5. Feature register

Every feature in v2.4, plus everything added on `main` up to 4 October, with what the code actually does.

Feature	Status	What's actually there	Next step
The night out			
Lock In (RSVP and ticket tiers)	Works	RSVP with tiers, saved through a checked server function (<code>upsert_rsvp_tier</code>).	None.
Touch Down (I'm here)	Works	Saved in <code>live_checkins</code> . The server marks it verified if the phone is within about 2 km of the venue. Failed check-ins are queued and resent when signal returns.	Tighten to a rotating door code plus 150 m (section 7.2).
Door QR scanning by staff	Works	QRCheckInScanner plus <code>secure_check_in</code> . Only the organiser, co-host or a scanner role can admit someone.	Needs signal. The QR never changes, so a screenshot works for anyone: move to signed passes (7.3).
Offline ticket cache	Partial	Your tickets are kept on the phone for 24 to 48 hours so they still display with no signal.	Display only. Doors can't verify them offline yet.
Ticket Vault ("cryptographic")	Demo only	The "signature" is the plain text <code>HMAC_ticket_event_time</code> , with no secret key. Any text starting with <code>HMAC_</code> passes the check. "Save to vault" creates a ticket for any event with no booking behind it.	Remove the "cryptographically vaulted" claim now. Replace with server-signed passes (7.3).
Ticket resale (face value)	Demo only	The listing is saved on the seller's phone only. The server save fails and the table doesn't exist. No money moves.	Needs payments (section 9) and a server ledger.
On Decks (who's playing)	Partial	The host sets the current artist or song, and a live bar updates for everyone (<code>event_now_playing</code>).	Add scheduled set times and a countdown to the next act.
Crowd meter	Partial	Anonymous crowd-level votes, averaged over the last 45 minutes (CrowdMeter). Its table (<code>event_crowd_votes</code>) isn't in any repo SQL file, so it only works if it was created on the live database by hand.	Add queue speed (Breeze, Moving, Packed).
Door queue thermometer	Demo only	Writes to <code>venue_door_queue</code> , which doesn't exist, and is cached on the phone.	Add the table with door-staff-only writes.
Carpool board	Partial	Offer or find a lift to an event (<code>event_carpools</code>), with live seat counts.	Match people by verified Resident building later (section 2).
Poster scan (OCR)	Works	Reads text off an uploaded flyer in the browser (Tesseract) and fills in the event form.	No server worker is needed yet.
Money			
Service bookings and escrow	Partial	Bookings go through checked server functions. No money is held: every booking stays 'unfunded' until a payment provider confirms payment.	Connect a payment provider (section 9).

Feature	Status	What's actually there	Next step
Table kitty and split bill	Demo only	Stored on one phone only, so your squad can't see it. No payment.	Needs payments.
Talent fee escrow	Demo only	Stored on the phone. "Releases on check-in" isn't implemented on the server.	Needs payments.
Promoter flyer bounties	Demo only	Stored on the phone. event_flyers and increment_flyer_claims aren't in any database file.	Add the tables and functions, then pay out through the ledger.
Business plans and boosts	Partial	Plan limits are enforced by the server. Upgrades aren't charged, so the prices shown are decorative.	Bill through the payment provider.
Gifting and coins	Partial	Turned off in launch mode. Cash-out is off because there's no payout rail.	After payments.
Flash Drops (paid push blasts)	Not started	—	Needs payments and marketing-consent opt-in (POPIA).
Identity and social			
Vibe Card	Works	Level, score, verified badge and crew. The share link was fixed on 27 September.	The level perks list promises features that don't exist yet.
Crossed paths and co-presence	Works	Reads real check-ins (live_checkins, public_profiles).	Needs the coordinates decision (7.4).
Proof of Sweat	Demo only	No microphone, step counter or Bluetooth is read. Built-in numbers (52 min loud music, 3,120 steps, 8 peers) make everyone "VERIFIED".	Drop it or rebuild it on real signals with consent (7.4).
Vibe Handshake, Where's My Crew	Demo only	Stored on the phone only (inPersonVibeService), with some random values.	Needs tables and real presence.
Booth and Street (DJ drops, track ID, kota tracker, car-meet decibels)	Demo only	Stored on the phone only (boothAndStreetService).	Pick the one or two worth building for real.
Sensory UI, haptics, face-down stealth, shake	Works	Uses the real accelerometer (expo-sensors) and haptics on phones.	Limited on the web.
Chats, crews and calls (Linked Up)	Not reviewed	—	—
Safety			
Safe Ride timer, Walk me to my car	Partial	Fixed 5 October: until then the panel claimed contacts and security were alerted, and nothing was sent. Now it says the truth and shares the trip with a friend through WhatsApp.	Real alerts need tables and a job that fires when time runs out, even with the phone off.
Emergency SOS	Not started	A function exists but no button uses it, and it writes to a table that doesn't exist.	Keep it unconnected until it really alerts someone.
Lost and found, shuttles, venue floor plan	Demo only	Sample data, stored on the phone.	Make it real per venue, or hide it.

Feature	Status	What's actually there	Next step
Food, sport, business			
The Meal (restaurants and specials)	Works	Posting, boosts and reach limits, enforced by the server since 27 September.	Run its SQL on the database.
Kota and food section, birthday radar, sports hub	Not reviewed	New on main (3 October).	—
Platform			
Shared login with The Resident	Works	One-time, single-use handoff codes bound to the target app (sso-redeem).	None.
Guardian monitoring	Works	GitHub Actions checks of the database, site and error log every 6 hours.	None.
Account deletion	Works	delete-account Edge Function plus purge.	Run account_deletion.sql.
POPIA location purge	Partial	Purge functions exist (90 days). The nightly schedule is commented out and pg_cron isn't enabled.	Enable it, and decide 48 hours or 90 days.
Live Activities, Dynamic Island, home widgets	Not started	—	Needs native apps (section 6).
Play Integrity / App Attest	Not started	—	Needs native apps.
WhatsApp promoter bot	Blocked	—	Needs a Meta WhatsApp Business Platform account.
Excellency Academy link	Not started	No Excellency code in this repository.	Where does Excellency live?

Why so many features are demos, and the fix pattern

1. **Saved on one phone only.** AsyncStorage never leaves the device, so a "squad kitty" is invisible to the squad.
2. **Server saves that can't work.** `supabase.from(...).insert(...).catch(...)` throws, because a Supabase query has no `.catch`. The surrounding `try` hides the error, so even a real table would never be written.
3. **Tables that don't exist.** `ticket_resales`, `venue_door_queue`, `talent_escrow`, `emergency_alerts`, `safety_pings`, `lost_and_found` and `event_flyers` aren't in any SQL file. Neither is the function `increment_flyer_claims`.

Fix pattern for each one: a migration (table plus row rules) → a checked server function → in the app, `const { error } = await ...` with the error shown to the user → a test. The Meal and the bookings fix from 27 September are working examples.

6. Widgets and glanceable surfaces

v2.4's widget catalog is the right idea. All of it is Not started, and most of it needs **native apps**, while production today is the website. So build each widget as an **in-app Live card first** (works everywhere, including the web), then the Android ongoing notification, then iOS Live Activities.

Widget	First version (in-app, all platforms)	Native version	Needs
Lineup and Stage Radar	Live card: on decks now, next act and countdown	Live Activity, Dynamic Island; Android ongoing notification	Scheduled set times (On Decks is Partial)
Door Watcher and Pass	Live card: queue time and your pass	Lock-screen activity	Real queue data; signed pass. Start it on Lock In or app open, not automatically within 1 km : automatic start needs "all the time" location.
Safe Crew and Ride Beacon	Safe Ride card (now honest, shares through WhatsApp)	Lock-screen activity with a "Home safe" button	Server alerts that fire when the phone is off
The Tonight Radar	Top 3 tonight on Radar	Home-screen widget	Native apps
Table Pitch-In tracker	Live card showing funding progress	Not needed	Payments (section 9)
Crowd Sense pulse	Prompt 15 minutes after Touch Down (CrowdMeter exists)	Notification action buttons	Queue-speed field

Native groundwork, in order

1. Publish Android (EAS production build) and add the ongoing-notification module.
2. Apple Developer account and an iOS build. Live Activities need a widget extension added through an Expo config plugin and a development build (not Expo Go).
3. App attestation: Play Integrity and App Attest, used by Touch Down (7.2).

7. Security, trust and privacy

7.1 Already in place

Database

- Row-level security on every table, tested in CI
- Admin self-promotion closed; money-minting functions locked
- Report brigading fixed: two new accounts can't auto-hide anyone
- Bookings, disputes and the wallet ledger are server-only, and unfunded bookings can't create balance
- The Meal's boost and view limits can't be set from the phone
- Unused privileged functions closed to clients

App and pipeline

- Share pages hide reported content
- Secret scanning blocks the build on a leaked key
- Edge Functions check the signed-in user explicitly
- Uploads re-checked for their real file type (stops stored XSS)
- External links opened safely (phishing checks)
- Safety panel no longer claims alerts it didn't send

7.2 Touch Down v2: proof of presence that holds up

Why v2.4's `record_touch_down` won't work as written: it uses tables that don't exist (`venues.location`, `touch_downs`, `active_event_nonces`). It never checks that someone is signed in. And the location and "accuracy" come from the phone, so checking them on the server doesn't stop someone faking their GPS. Distance is useful evidence, but it isn't proof.

L3

Scanned at the door by staff. Exists today (`QRCheckInScanner` + `secure_check_in`). The strongest proof.

L2

Rotating door code (new). A screen at the door shows a 6-digit code or QR that changes every 30 seconds. The phone sends it with the check-in, and the server checks it against the event's secret. You can't fake it from home. This is the real anti-spoofing step.

L1

GPS within 150 m, tightened from about 2 km today, plus app attestation once native. Counts, but weighs less.

L0

Unverified. Still allowed (visibility is a safety feature), but it never earns score or trending weight.

```
-- sketch: L2 check, on the real table (live_checkins)
create table event_door_secrets (event_id uuid primary key references events on delete cascade,
                                secret bytea not null);           -- no client access at all
create function touch_down_with_code(p_event uuid, p_code text, p_lat float8, p_lon float8)
returns jsonb security definer set search_path = public language plpgsql as $$
begin
  if auth.uid() is null then raise exception 'not signed in'; end if;
  -- accept the current 30-second window and the one before it (clock drift)
  if not door_code_valid(p_event, p_code, now()) then
    return jsonb_build_object('ok', false, 'reason', 'code expired');
  end if;
  insert into live_checkins (user_id, event_id, lat, lon, verified, verify_level)
  values (auth.uid(), p_event, round(p_lat::numeric, 3), round(p_lon::numeric, 3), true, 2)
  on conflict (user_id, event_id) do update set verify_level = greatest(live_checkins.verify_level, 2);
  return jsonb_build_object('ok', true, 'level', 2);
end $$;
```

7.3 Offline "Pocket Pass": signed properly

Today's vault "signature" is plain text that anyone can type. A real offline pass works like this:

- 1. **The server signs** (Edge Function) with an Ed25519 private key that never leaves the server. The pass holds the ticket, event, holder, tier and expiry.
- 2. **Door scanners hold only the public key**, synced with the event's revocation list before doors open. They verify with no signal.
- 3. **Screenshots don't work**: the pass shows a short code that rotates every 30 seconds, derived from a per-ticket secret the scanner can check. An animated watermark alone doesn't stop screen recording.
- 4. **Resale is done by the server**: it revokes the seller's pass and issues a new one only after payment clears (section 9).
- 5. **The phone stores the pass in secure storage** (Expo SecureStore), not AsyncStorage.

7.4 Privacy and POPIA

Topic	Today → recommendation
Exact check-in coordinates	Stored, and readable by any signed-in user (the map uses them). → Store venue-level presence, or round to about 100 m, and show "at Venue X". Decision 4.
Retention	Purge set to 90 days, but not scheduled. v2.4 says 48 hours. → Choose (decision 5), enable pg_cron, and confirm it runs with Guardian.
Proof of Sweat	Proposes the microphone (85 dB), steps and Bluetooth peers. → Microphone use needs explicit consent and store disclosures, and drains the battery. Recommend dropping it in favour of Touch Down L2 and L3.
Cross-app data	→ Opt-in per purpose (rides, perks, leads), recorded with a timestamp, and withdrawable.
Marketing pushes (Flash Drops)	→ Separate opt-in for promotional messages; unsubscribe in one tap.

7.5 Two v2.4 proposals to change

Cross-app tokens (aud "gruvs_api")

One shared Supabase project issues the same kind of token for every app, so custom audiences aren't available. Keep what's built: row-level rules per app, plus app-bound one-time login codes.

Instagram flyer scraping

Against Instagram's terms: it risks account bans and legal claims, and the data is unreliable. Use what promoters send: upload or WhatsApp-forward a flyer, read in the browser (exists), with one-tap admin approval.

8. Database and operations

How database changes ship now

Pending changes live in `supabase/migrations/` as **11 numbered files**. CI builds the schema, applies every migration twice to prove each is safe to re-run, and runs the security tests. To ship: add the GitHub secret `SUPABASE_PRODUCTION_DB_URL`, then run the Deploy workflow by hand. Steps are in `DB_UPDATE_RUNBOOK.md`.

Owner checklist

#	Action	Why
1	Restore the Supabase project	The app has no data without it
2	Merge the branch into main (open the pull request)	21 commits of fixes and docs
3	Run the migrations (Deploy workflow, or paste the files in order)	Security and integrity fixes
4	Run <code>admin_grants.sql</code> with the Google account's email	The two admin accounts
5	Enable <code>pg_cron</code> , then run the retention and maintenance files	POPIA purge, clean-up
6	Run <code>the_meal.sql</code> and <code>account_deletion.sql</code> if the contract check lists them	The Meal, store compliance
7	Rotate the Spotify client secret	It was once exposed in builds
8	Delete the Vercel projects; close PR #28	Failed-build noise
9	Move to the paid Supabase tier at launch	No pausing; daily backups

Known gaps

- **17 app features** call database functions that no repo file defines. Each is a feature that does nothing, with no error. Run `APP_DB_CONTRACT_CHECK.sql` on the restored database to see which actually exist.
- **7 tables and 1 function** used by the new `main` features don't exist (section 5).
- **No snapshot of the live schema yet.** Take one with `supabase db pull` once restored, so a rebuild can't lose what was applied by hand.

9. Money and payments

One decision unlocks seven features

Escrow, Table Pitch-In, the table kitty, ticket resale, Flash Drops, business plan billing and cash-out all need a **payment provider**. Until there is one, none of them should show prices as if money is moving.

Principles

1. **The provider holds the money, not The Gruvs.** Holding customers' money yourselves can make you a regulated payments business. Use the provider's split-payment or marketplace product. Confirm with the provider and a lawyer.
2. **Only the server writes money.** The ledger (`wallet_transactions`) is append-only and clients can't write to it Works.
3. **Payment is confirmed by the provider's webhook**, never by the phone. The bookings table already waits for `payment_status = 'captured'`, which only the server can set Works.
4. **Fees are taken in the split** (for example 3% on Pitch-In), shown before paying, and itemised on receipts.

Candidate providers

South African options include **Paystack**, **Peach Payments**, **PayFast** and **Ozow**. Some of them offer Capitec Pay and instant EFT. Compare split-payment support, payout speed, fees, and how refunds and disputes work.

Feature	Status	Once payments exist
Service escrow	Partial	The webhook marks captured; release pays the provider through the ledger. The plumbing is ready.
Table Pitch-In / kitty	Demo only	Up to 6 payers fund one table; the table pass is issued when fully funded; refunds if it isn't.
Ticket resale	Demo only	Face-value cap enforced by the server; pass reissued on payment (7.3).
Flash Drops	Not started	Prepaid; sent only to people who opted in to promotions.
Business plans	Partial	Monthly billing; the plan is set by the webhook, never the app.
Cash-out	Partial	Last: needs identity checks and payout limits.

10. Discovery and SEO

Today: the sitemap lists 4 static pages plus upcoming events. When Google opens a share link, it gets a logo, one sentence and an instant redirect, so **there is almost nothing to rank**. The fix is real pages first, then a rich sitemap.

Phase	What	Result
0	Keep the generated events catalog out of Google	Avoids a spam penalty for the whole domain
1	Full event and profile pages with structured data (Event, Place, Offer in ZAR, performers), a sitemap index, built live and cached hourly	Events can appear in Google's event results
2	City, category, city + category ("amapiano Johannesburg"), venue and "this weekend" pages	The pages people actually search for
3	The Meal, series and tours, sports, reels, recaps	Food, sport and video searches
4	City guides, IndexNow, generated preview images, more languages	Evergreen content, faster indexing

Quality rules

- A city, category or venue page exists only with at least 3 real upcoming events. Otherwise it stays out of Google.
- Never listed: private or crew-only events, reported content, non-discoverable profiles, deleted accounts.
- Ratings and "going" counts come only from real reviews and real RSVPs.
- Profiles on Google: organisers, artists and businesses only (recommended, decision 7).

The generated events catalog

`public/data/events-catalog.json` (formerly `globalEventsCatalog.js`) holds 870 events built from template titles, real venue names and stock photos, and labels them "real authentic events". They aren't happening. Shown as real, they send people to real venues on nights with no event. In Google they'd break its spam rules. **Label them as examples or remove them** (decision 3), and never list them in the sitemap.

11. Growth and revenue

Cold start (weeks 1 to 8)

Lever (v2.4)	Readiness	Note
The Bouncer Trojan Horse	Works	The door scanner and guest list already exist. Pick 5 venues, give their door staff the scanner role, and guests claim passes in the app. The strongest lever.
Residence corridor takeovers	Partial	Needs The Resident live in 2 or 3 buildings, plus the carpool board.
Editorial seeding	Partial	Must be real events from promoters and venues, not the generated catalog.
Flyer intake	Partial	In-browser OCR exists. Add the WhatsApp forwarding bot once there's a Meta Business account.

Revenue streams

Stream	Status	Depends on
Promoter Flash Drops (about R500 per blast)	Not started	Payments and promotional opt-in
Pitch-In convenience fee (2.5 to 3.5%)	Blocked	Payments
Business plans and boosts	Partial	Billing
The Meal boosts	Works	Free today; paid boosts need billing
Ecosystem leads (Resident, Excellency)	Not started	Explicit cross-app consent (7.4)

12. Roadmap

Phase 0 · Unblock

This week

- Restore Supabase · merge the branch · run the migrations and the owner checklist (section 8)
- Hide money, ticket and safety demos behind `feature()` flags; label the rest "Preview"
- Decide what happens to the generated catalog

Phase 1 · Trust

Weeks 1 to 3

- Touch Down v2: rotating door code (L2), 150 m GPS (L1), verify levels in scoring
- Venue-level coordinates; POPIA purge scheduled and monitored
- Make the demos real one at a time with the fix pattern (section 5), starting with the door queue and the server-side Safe Ride alert
- Resolve the 17 missing functions · publish on Google Play

Phase 2 · The live night

Weeks 4 to 6

- Navigation becomes Radar · Live · Passport, with context-aware Live mode
- In-app Live cards: On Decks with set times and countdown, queue speed, crew status
- Signed Pocket Pass (7.3) replacing the text "vault"
- SEO phases 1 and 2: real event, city and category pages, plus a sitemap index

Phase 3 · Money

Weeks 7 to 9

- Payment provider integration and webhooks → escrow capture
- Table Pitch-In, ticket resale, Flash Drops, business plan billing · cash-out last

Phase 4 · Native and ecosystem

Weeks 10 to 12 and beyond

- iOS app · Android ongoing notifications · iOS Live Activities and widgets · app attestation
- Resident ride pooler and door perks · Excellency Sunday Reset and credits (if Excellency is ready)
- WhatsApp promoter bot · pilot with 5 anchor venues in Braamfontein and Rosebank

13. Decisions needed from you

#	Decision	Recommendation
1	Demo features	Hide the ticket, money and safety demos until they're real; label the fun ones (sensory, Booth and Street) "Preview".
2	Payment provider	Compare Paystack, Peach Payments, PayFast and Ozow on split payments and fees, then pick one.
3	Generated events catalog	Label as examples, or remove. Never in the sitemap.
4	Who sees check-in coordinates	Venue-level only.
5	Location retention	48 hours for exact points, if they're kept at all; aggregate counts kept longer.
6	SEO URL style	Keep <code>/share/...</code> (no server change), or cleaner paths (needs nginx changes).
7	Profiles on Google	Organisers, artists and businesses only.
8	Proof of Sweat	Drop the microphone and Bluetooth measures; rely on Touch Down levels.
9	Excellency	Where does its code live? The ecosystem features need its server.
10	Level perks on the Vibe Card	Build them, or mark the unbuilt ones "Coming soon".

14. Appendix: work completed on this branch

Branch `claude/future-failure-analysis-df20ej`, 21 commits ahead of `main` (including this document). Each fix has a test unless noted.

Area	Fixed
Privileges	Four privileged database functions could mint money or delete any row; the admin-role guard never fired; vibe counts could be forged for other users. All closed and tested.
Moderation	Two fresh accounts could auto-hide anyone. Reports are now weighted by account trust and age; one report per person per target.
Money	"Escrow" created balance from nothing. Bookings and disputes now go through the server only; the ledger is append-only; unfunded bookings credit nothing.
The Meal	Owners could boost themselves and reset view counts directly. Locked.
Unused functions	4 privileged functions callable by anyone, even logged out (one leaked any user's unread count). Closed.
Migrations	New <code>supabase/migrations/</code> ; CI applies each twice and runs 5 security tests; the deploy pipeline can push them. Two broken paste-in files fixed.
Pipeline	Secret scanning had never run; the Spotify secret was plumbed into builds; share pages bypassed moderation; Edge Function auth tightened.
Sign-in	Users with the right password were told it was wrong (unconfirmed email); clearer errors; resend-confirmation button.
Vibe Card	Every shared card linked to a page that didn't exist; a profile-load failure could send the user's email as their name.
Crashes on main	Missing imports in <code>EventDetailScreen</code> ; <code>Explore</code> and <code>Profile</code> crashed after the responsive overhaul (<code>width</code> undefined).
Safety (5 Oct)	The Night Safety panel claimed contacts and security were alerted when nothing was sent. Now truthful, with WhatsApp sharing that reaches a real person.
Scale	109 cascading foreign keys indexed (account deletion stays within time limits); the follower list is no longer shipped to every phone.

Generated 2026-10-05 from `docs/master-spec/build.py`. To update: edit the data in that file and re-run `python3 docs/master-spec/build.py && node docs/master-spec/render.mjs`.